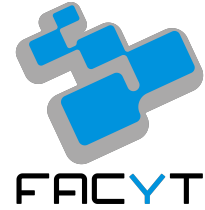




Universidad de Carabobo
Facultad Experimental de Ciencias y Tecnología
Arquitectura del Computador
Sección 1



Informe

Práctica de Laboratorio 1

Integrantes:

Penélope Robledo, C.I. 32.482.938

Nathalia Noguera, C.I. 32.756.046

Profesor:

José Canache

Naguanagua, julio de 2026

1. ¿Cómo se implementa la recursividad en MIPS32 ¿Qué papel cumple la pila (\$sp)?

La recursividad es una técnica de programación en la que una subrutina se invoca a sí misma para resolver un problema mediante la división del mismo en subproblemas menores. En este caso, para determinar la paridad de un número entero n , la definición recursiva es:

- Caso base: Si $n=0$, el resultado es 0.
- Caso recursivo: Si $n>0$, el resultado es 1 paridad($n - 1$).

Sin embargo, para la ejecución en código, es necesario tener en cuenta el tipo de lenguaje de programación, aquellos de alto nivel (como C, Python o Java), el compilador automatiza la gestión de la recursividad, el programador solo declara la función y el compilador gestiona internamente el flujo de ejecución, el almacenamiento temporal de las variables locales y el retorno al punto de llamada.

En cambio, en el lenguaje ensamblador MIPS32, este proceso no es automático. MIPS32 es un lenguaje de bajo nivel que requiere que el programador controle explícitamente el flujo de ejecución y el almacenamiento de datos, como no existen construcciones predefinidas para la recursividad, el programador debe implementar manualmente la gestión de registros y memoria que normalmente realiza un compilador. Para implementar la recursividad en MIPS32, es obligatorio utilizar tres elementos clave:

- Instrucción jal (Jump and Link): Esta instrucción transfiere el flujo de ejecución a la etiqueta de la función. Al mismo tiempo, guarda automáticamente la dirección de la siguiente instrucción en el registro \$ra (Return Address).
- Registro \$ra: Contiene la dirección de retorno, es fundamental porque indica al procesador a qué punto del código debe volver cuando finalice la subrutina.
- La Pila (Stack): Es una estructura de datos en memoria organizada bajo el principio LIFO (Last In, First Out). El registro \$sp (Stack Pointer) indica la cima de la pila.

Otro concepto clave en el entendimiento de la recursividad es la sobrescritura, cuando una función se llama a sí misma, la instrucción jal sobrescribe el valor actual en el registro \$ra con la nueva dirección de retorno, si no se protege el valor original del \$ra antes de la llamada recursiva, la dirección de retorno anterior se pierde, lo que impide que el programa regrese correctamente

a los niveles superiores de la recursión. Para evitar esto, cada nivel de la recursión debe preservar su propio estado, entrando así el papel de la pila.

Para preservar el estado, se crea un Stack Frame (marco de pila) por cada llamada recursiva, este marco es un espacio reservado en la memoria donde se guardan los datos necesarios para que la función pueda retomar su ejecución después de que la llamada recursiva retorne. Los elementos que se deben guardar en la pila antes de cada llamada recursiva son:

- Dirección de retorno (\$ra): Para saber a dónde volver.
- Argumentos de la función (\$a0): El valor de n, ya que se modificará para la siguiente llamada.
- Registros temporales: Cualquier otro registro que la función utilice y deba preservar.

Secuencia de ejecución para la recursividad:

El proceso técnico que debe seguir el programador para realizar una llamada recursiva es el siguiente:

1. Reserva de memoria: Se decrementa el puntero de pila (\$sp) para crear espacio (ejemplo: addi \$sp, \$sp, -8).
2. Resguardo de datos: Se almacenan los valores actuales (\$ra y \$a0) en la pila utilizando la instrucción sw (store word).
3. Actualización del parámetro: Se modifica el registro \$a0 (se resta 1 a n) para preparar el argumento de la siguiente llamada.
4. Llamada recursiva: Se ejecuta jal a la etiqueta de la función.
5. Restauración: Cuando la función retorna, se cargan los valores guardados anteriormente desde la pila hacia los registros (\$ra y \$a0) utilizando lw (load word).
6. Liberación de memoria: Se incrementa el puntero de pila (\$sp) para liberar el espacio reservado (ejemplo: addi \$sp, \$sp, 8).
7. Retorno: Finalmente, se ejecuta jr \$ra para volver a la dirección de retorno recuperada.

Este ciclo se repite hasta alcanzar el caso base ($n=0$), momento en el cual la pila comienza a liberarse y el control del programa regresa secuencialmente, esto se realiza de la siguiente forma:

1. Finalización del Caso Base: Cuando se alcanza el caso base, la función ya no realiza más saltos

recursivos. En su lugar, ejecuta el bloque de restauración y libera el espacio de su propio stack frame. Al ejecutar `jr $ra`, el control regresa a la función que la llamó inmediatamente antes.

2. Recuperación del contexto previo: Al retomar el control, cada nivel de la función se encuentra exactamente en la instrucción que seguía al `jal` original. Como cada nivel preservó su propio `$ra` y sus variables en la pila, al ejecutar las instrucciones de Restauración (`lw`), el programa recupera la memoria "de lo que estaba haciendo antes de pedir la siguiente llamada recursiva.

3. Ejecución en cascada: Este proceso ocurre de forma secuencial y ordenada debido a la estructura LIFO de la pila. El nivel de profundidad más reciente es el primero en ser atendido, restaurando sus valores y terminando su ejecución, al cerrar ese marco, el control vuelve al nivel anterior, que ahora tiene sus valores restaurados y puede continuar con su propio cálculo.

4. Conclusión de la cadena: Este ciclo de recuperación se repite hacia atrás, nivel por nivel, hasta que la primera llamada a la función (la que se ejecutó desde el programa principal) recupera su contexto original. En ese momento, la pila ha quedado completamente liberada, los registros han sido restaurados y el programa principal recibe el resultado final del cálculo recursivo.

2. ¿Qué riesgos de desbordamiento existen? ¿Cómo mitigarlos?

Uno de los principales riesgos asociados a la implementación de funciones recursivas en MIPS32 es el desbordamiento de pila (stack overflow), este riesgo surge debido a la naturaleza misma de la recursividad y a las limitaciones de los recursos de memoria disponibles en el sistema.

El desbordamiento de pila ocurre cuando el número de llamadas recursivas es tan grande que se agota el espacio reservado para la pila en la memoria, cada llamada recursiva genera un nuevo marco de pila (stack frame) que ocupa una cantidad determinada de bytes (por ejemplo, 8, 12 o más, dependiendo de los registros que se guarden). Si la función no alcanza el caso base con suficiente rapidez, la pila continúa creciendo hacia direcciones más bajas hasta que \$sp sobrepasa los límites asignados a la región de pila.

Entre las consecuencias de este desbordamiento están:

- Sobrescritura de otras áreas de memoria (datos, código, etc).
- Comportamiento impredecible del programa.
- Caída del sistema o excepciones en simuladores y entornos reales.
- En casos extremos, corrupción de la dirección de retorno (\$ra), lo que genera saltos a direcciones inválidas.

En la función de paridad recursiva descrita, el riesgo es particularmente relevante cuando se trabaja con valores grandes de n , aunque el ejemplo de paridad suele tener una profundidad igual a n (lo que en la práctica es manejable para números pequeños), en problemas recursivos mayores, el riesgo aumenta considerablemente.

Factores que Incrementan el Riesgo:

- Profundidad de recursión elevada: Mientras mayor sea el valor inicial de n , mayor será el número de marcos de pila creados.
- Tamaño de cada marco de pila: Cuantos más registros se guarden en cada llamada (valores de \$ra, \$a0, registros \$s, etc.), más rápido se consume el espacio disponible.
- Limitaciones del entorno: Los simuladores como MARS tienen un tamaño de pila predefinido relativamente pequeño, en sistemas embebidos reales con MIPS32, la memoria asignada a la pila puede ser aún más restringida.

- Ausencia de caso base o caso base inalcanzable: Si la condición de terminación nunca se cumple, la recursión se vuelve infinita y el desbordamiento es seguro.

Estrategias para Mitigar los Riesgos

Existen varias técnicas que permiten reducir o eliminar el riesgo de desbordamiento de pila:

- Verificación explícita del parámetro de entrada: Antes de iniciar la recursión, se debe validar que el valor de n se encuentre dentro de un rango seguro. Por ejemplo, rechazar o limitar valores excesivamente grandes.
- Optimización del tamaño del marco de pila: Guardar únicamente los registros estrictamente necesarios en cada llamada. Minimizar el uso de registros preservados y reducir al mínimo el espacio reservado con `addi $sp, $sp, -N`.
- Conversión a versión iterativa: Cuando sea posible, transformar la solución recursiva en un bucle simple. En el caso de la paridad, una implementación iterativa (usando un ciclo que reste 1 repetidamente o mediante operaciones bitwise) elimina completamente el uso de la pila y el riesgo de desbordamiento.
- Uso de recursión de cola: Reorganizar el código para que la llamada recursiva sea la última instrucción de la función. Aunque MIPS32 no realiza optimización automática de recursión de cola, esta estructura facilita una posible conversión manual a iteración.
- Aumento del tamaño de la pila (cuando sea posible): En simuladores o sistemas donde se pueda configurar, ampliar la región de memoria asignada a la pila.
- Monitoreo del uso de la pila: Durante el desarrollo, se puede inspeccionar el valor del registro `$sp` en el simulador para verificar que no se acerque a los límites inferiores de la memoria asignada.

3. ¿Qué diferencias encontraste entre una implementación iterativa y una recursiva en cuanto al uso de memoria y registros?

La elección entre un enfoque recursivo e iterativo para resolver un mismo problema, genera diferencias significativas en el uso de memoria y en la gestión de registros. A continuación se detallan estas diferencias de manera comparativa.

1. Uso de Memoria

Implementación Recursiva: En la versión recursiva, el consumo de memoria es directamente proporcional a la profundidad de la recursión, cada llamada a la función genera un marco de pila (stack frame) independiente. En el caso de la función de paridad, por cada valor de n se reserva espacio en la pila (generalmente 8 o 12 bytes) para almacenar al menos el registro $\$ra$ y el parámetro $\$a0$. Por lo tanto, para un valor inicial n , se crean aproximadamente n marcos de pila, esto implica un consumo de memoria $O(n)$ en el espacio de pila. Si n es muy grande, existe un riesgo real de desbordamiento de pila, ya que el espacio disponible para la pila es finito tanto en simuladores como en sistemas embebidos.

Implementación Iterativa: En una versión iterativa, el consumo de memoria es constante ($O(1)$), y también, no se utiliza la pila para almacenar estados de llamadas sucesivas, ya que no existen llamadas recursivas. El algoritmo emplea un bucle que modifica el valor del parámetro con un contador en cada iteración hasta alcanzar la condición de terminación. Solo se reserva una cantidad fija de memoria para variables locales o registros temporales, independientemente del valor de n , esto elimina por completo el riesgo de desbordamiento de pila relacionado con la profundidad del algoritmo.

2. Uso de Registros

Implementación Recursiva:

La versión recursiva requiere un uso más intensivo y cuidadoso de los registros debido a la necesidad de preservar el estado entre llamadas. Es obligatorio:

- Guardar el registro $\$ra$ en cada llamada para poder retornar correctamente.
- Guardar el parámetro de entrada ($\$a0$) si se necesita después de la llamada recursiva.
- Preservar registros $\$s0$ - $\$s7$ si la función los utiliza como variables locales.

- Utilizar registros temporales (\$t0-\$t9) para cálculos intermedios.

Esto obliga al programador a realizar operaciones frecuentes de almacenamiento (sw) y recuperación (lw) en la pila, aumentando el número de instrucciones y el tiempo de ejecución.

Implementación Iterativa:

La versión iterativa suele requerir menos registros y un manejo más simple. Típicamente se utilizan:

- Un registro para el parámetro o contador (\$a0 o un registro \$t).
- Uno o dos registros temporales (\$t0, \$t1) para realizar las operaciones de resta, comparación o alternancia de paridad.
- El registro \$v0 únicamente al final para devolver el resultado.

No es necesario guardar \$ra repetidamente ni realizar operaciones de pila en cada iteración, el flujo del programa permanece dentro de la misma función, por lo que los registros mantienen su valor a lo largo del bucle.

Aunque la implementación recursiva resulta más intuitiva a la definición matemática del problema, consume más memoria y requiere una gestión más compleja de registros y de la pila. Por su parte, la implementación iterativa es considerablemente más eficiente en el uso de recursos, especialmente en cuanto a memoria y velocidad de ejecución, lo que la hace preferible en aplicaciones donde el rendimiento y la limitación de recursos son factores críticos. En el contexto de MIPS32, la versión iterativa suele ser la opción recomendada cuando no existen requisitos específicos que justifiquen el uso de recursividad.

4. ¿Qué diferencias encontraste entre los ejemplos académicos del libro y un ejercicio completo y operativo en MIPS32?

Al comparar los ejemplos presentados en los libros de texto con una implementación completa y ejecutable en MIPS32, se observan diferencias importantes que reflejan el paso de un enfoque teórico a uno práctico y operativo. A continuación se detallan las principales diferencias identificadas:

1. Estructura General del Programa

Los ejemplos académicos de los libros suelen mostrar únicamente el fragmento de código relevante de una función o rutina, sin incluir la estructura completa de un programa MIPS32. En este tipo de ejercicios, se omiten directivas esenciales como:

- `.text` para indicar la sección de código ejecutable.
- `.globl main` para declarar el punto de entrada del programa.
- `.data` para definir variables o constantes en memoria de datos (aunque en muchos casos no sean necesarias).

En cambio, un ejercicio completo y operativo en MIPS32 requiere obligatoriamente estas directivas para que el ensamblador y el simulador (como MARS) puedan procesar correctamente el archivo. Además, debe incluir una sección `main` que inicialice los parámetros, realice la llamada a la función y gestione la terminación del programa mediante llamadas al sistema (`syscalls`), así, esta estructura completa asegura que el programa sea autónomo y ejecutable.

2. Nivel de Detalle y Completitud

En los libros, los ejemplos suelen ser fragmentos aislados de código que ilustran un concepto específico (por ejemplo, el uso de la pila o una llamada recursiva), estas partes con frecuencia asumen que ciertos registros ya contienen valores válidos o que el entorno de ejecución ya está configurado.

Por el contrario, una implementación operativa en MIPS32 debe considerar; la inicialización explícita de todos los valores necesarios, el manejo completo del prólogo y epílogo de la función

(reserva y liberación de pila, guardado y restauración de registros), la correcta devolución de resultados mediante el registro \$v0 y la terminación ordenada del programa. Generando un código más largo y detallado, pero también más realista y funcional.

3. Posibilidad de Ejecución y Verificación

Una diferencia fundamental radica en la naturaleza estática de los ejemplos del libro frente a la naturaleza dinámica de un ejercicio en MIPS32. Mientras que en el libro el lector solo puede leer y analizar teóricamente el código, en MARS es posible:

- Ensamblar y ejecutar el programa paso a paso.
- Observar en tiempo real el contenido de los registros (\$sp, \$ra, \$a0, \$v0, etc.).
- Monitorear el crecimiento y decrecimiento de la pila en el Data Segment.
- Verificar el resultado final mediante syscalls.
- Detectar errores de programación (como desbordamiento de pila o manejo incorrecto de registros) de forma práctica.

Esta capacidad de ejecución proporciona una comprensión mucho más profunda del comportamiento real de la arquitectura.

4. Reutilización y Modularidad del Código

Los ejemplos académicos suelen presentar rutinas aisladas que pueden ser reutilizadas o citadas parcialmente en diferentes secciones del libro, en cambio, en un ejercicio completo, se debe escribir un programa coherente que integre todas las partes: inicialización, llamada a la función, manejo de resultados y finalización. Aunque se puede reutilizar código de ejemplos previos, es necesario adaptarlo al contexto general del programa, ajustando etiquetas, registros y convenciones de llamada.

5. Manejo de Convenciones y Buenas Prácticas

Los libros a menudo simplifican o omiten ciertas convenciones de llamada de MIPS32 (como el uso estandarizado de los registros \$a0-\$a3, \$v0-\$v1, \$s0-\$s7, \$t0-\$t9 y el manejo del stack

pointer), a diferencia de una implementación operativa, ya que es indispensable respetar estas convenciones para garantizar la compatibilidad y el correcto funcionamiento, especialmente en programas que combinan múltiples funciones o llamadas recursivas.

5. Elaborar un tutorial de la ejecución paso a paso en MARS.

Tutorial de Ejecución Paso a Paso en MARS del Algoritmo Iterativo

A continuación se presenta un tutorial detallado sobre la ejecución del algoritmo iterativo de paridad en el simulador MARS.

Preparación en MARS

- Se abre el simulador MARS y se crea un nuevo archivo en el que se copia el código completo de la versión iterativa.
- Se presiona Assemble para ensamblar el programa.
- Se verifica en la ventana inferior "Messages" que no existan errores de ensamblado.
- Se activan las siguientes vistas necesarias para seguir la ejecución:
 - Registers (ventana superior derecha, muestra el estado de los registros).
 - Text Segment (ventana central, muestra el código fuente con direcciones).
 - Data Segment (ventana inferior, se selecciona la pestaña "Data Segment" para ver las cadenas de texto).
 - Messages (ventana inferior, muestra las salidas del programa).

Ejecución Paso a Paso

La ejecución se realiza con la tecla Step para avanzar instrucción por instrucción. En la sección main, el programa ejecuta una syscall (li \$v0, 4) para imprimir el mensaje "Ingrese un numero entero positivo:". Posteriormente, con li \$v0, 5 y syscall, lee un número entero ingresado por el usuario y lo almacena en \$v0. Este valor se transfiere a \$a0 mediante move \$a0, \$v0. A continuación se ejecuta jal paridad, lo que transfiere el control a la función paridad. Dentro de la función paridad:

- Se inicializa el registro \$v0 con el valor 0 (li \$v0, 0).
- Se entra al bucle etiquetado como ciclo.
- Mientras el contenido de \$a0 no sea igual a cero (beq \$a0, \$zero, fin), se realizan las siguientes operaciones:
 - Se carga el valor 1 en el registro \$t0.

- Se ejecuta sub \$v0, \$t0, \$v0, que alterna el valor de \$v0 entre 0 y 1 en cada iteración.
- Se decrementa el valor de \$a0 en una unidad (addi \$a0, \$a0, -1).
- Se regresa al inicio del bucle mediante j ciclo.
- Cuando \$a0 alcanza el valor cero, se sale del bucle y se ejecuta jr \$ra, retornando el control a la sección main.

De regreso en main, el resultado contenido en \$v0 se copia al registro \$t0, Se imprime el mensaje Resultado: "mediante syscall. Luego, según el valor de \$t0:

- Si \$t0 es igual a cero, se imprime .^{El} numero es par."
- Si \$t0 es igual a uno, se imprime .^{El} numero es impar."

Finalmente, se imprime un salto de línea y se termina la ejecución del programa con li \$v0, 10 y la syscall correspondiente.

Tutorial de Ejecución Paso a Paso en MARS del Algoritmo Recursivo:

A continuación se presenta un tutorial detallado sobre la ejecución del algoritmo recursivo de paridad en el simulador MARS.

Preparación en MARS

- Se abre el simulador MARS y se crea un nuevo archivo en el que se copia el código completo de la versión iterativa.
- Se presiona Assemble para ensamblar el programa.
- Se verifica en la ventana inferior "Messages" que no existan errores de ensamblado.
- Se activan las siguientes vistas necesarias para seguir la ejecución:
 - Registers (ventana superior derecha, muestra el estado de los registros).
 - Text Segment (ventana central, muestra el código fuente con direcciones).
 - Data Segment (ventana inferior, se selecciona la pestaña "Data Segment" para ver las cadenas de texto).

- Messages (ventana inferior, muestra las salidas del programa).

Ejecución Paso a Paso

La ejecución se realiza con la tecla Step para avanzar instrucción por instrucción y poder observar el comportamiento de la pila. En la sección main, el programa ejecuta una syscall (li \$v0, 4) para imprimir el mensaje "Ingrese un numero entero positivo:". Posteriormente, con li \$v0, 5 y syscall, lee un número entero ingresado por el usuario y lo almacena en \$v0. Este valor se transfiere a \$a0 mediante move \$a0, \$v0.

A continuación se ejecuta jal paridad, lo que transfiere el control a la función paridad. Dentro de la función paridad:

- Se reserva espacio en la pila con addi \$sp, \$sp, -8.
- Se almacenan el parámetro \$a0 y la dirección de retorno \$ra en la pila mediante sw \$a0, 0(\$sp) y sw \$ra, 4(\$sp).
- Se evalúa el caso base mediante siti \$t0, \$a0, 1. Si el resultado es distinto de cero (beq \$t0, \$zero, caso_recurso), se salta al caso recursivo. De lo contrario, se ejecuta el caso base: se carga 0 en \$v0, se libera la pila con addi \$sp, \$sp, 8 y se retorna con jr \$ra.

En el caso recursivo:

- Se decrementa el argumento actual con addi \$a0, \$a0, -1.
- Se realiza la llamada recursiva mediante jal paridad. Aquí es donde se observa el crecimiento de la pila: por cada jal, se crea un nuevo stack frame que preserva el estado del nivel anterior.

Tras el retorno de la llamada, el programa ejecuta la restauración: se cargan los valores guardados con lw \$a0, 0(\$sp) y lw \$ra, 4(\$sp), y se libera el espacio en memoria con addi \$sp, \$sp, 8. Finalmente, se realiza la lógica de la paridad: se carga el valor 1 en \$t1 y se ejecuta sub \$v0, \$t1, \$v0, lo cual invierte el resultado obtenido del nivel inferior. El control vuelve al nivel superior mediante jr \$ra.

Al finalizar la última llamada, el registro \$v0 contendrá el resultado final, el cual es recuperado por la sección main para imprimir si el número es par o impar. Durante este proceso, si observa la ventana "Registers" y la dirección de memoria apuntada por \$sp, podrá notar cómo la pila crece y decrece de forma dinámica hasta completarse.

6. Justificar la elección del enfoque (iterativo o recursivo) según eficiencia y claridad en MIPS.

En primer lugar, si bien ambos enfoques ofrecen el correcto resultado de la evaluación de la paridad de un número con una diferencia de tiempo casi imperceptible para el usuario con valores pequeños, una versión del algoritmo cuenta con ciertos beneficios sobre la otra, contrastando la claridad y similitud a la expresión en forma matemática con lo que está ocurriendo realmente dentro del hardware en cuanto al manejo de recursos y consecuentemente la eficiencia.

Con una implementación apegada a la fórmula que describe el problema, con un caso base y caso recursivo equivalentes a la definición matemática de la paridad de acuerdo con el valor de n , el algoritmo recursivo otorga una ventaja en su codificación al ser más intuitivo a nivel conceptual o teórico. No obstante, debido a la naturaleza y sintaxis del lenguaje ensamblador MIPS32, el manejo de la pila con creación del marco de pila o stack frame, además de las instrucciones de carga y almacenamiento (`lw` y `sw`) que deben ir a la par con este, pueden llegar a opacar este beneficio ya que al carecer de un compilador que maneje el flujo de manera independiente, se deben realizar todas estas operaciones manualmente con cautela para que la recursión no genere errores y no se pierdan valores como el n actual y la dirección de retorno entre llamadas. Todo esto converge en que la atención esté dirigida principalmente a la gestión de registros, pila y memoria en lugar de la lógica del algoritmo.

Dicha complejidad no se presenta en el enfoque iterativo, en el que las operaciones tienen un valor constante y el uso de la pila no es requerido, resolviendo completamente el problema en un único ciclo y función con una lógica a su vez sencilla, que aporta claridad y facilidad al momento de realizar un posterior mantenimiento al algoritmo.

Por otra parte, utilizando como criterio de comparación la eficiencia, esta puede evaluarse en cuanto el uso de memoria y el tiempo de ejecución. En el apartado de consumo de memoria, la elección del enfoque iterativo es de nuevo altamente recomendable. En la versión recursiva, la memoria utilizada en la pila corresponde linealmente a la cantidad de llamadas llevadas a cabo ($O(n)$).

Esto ocasiona que en casos donde el usuario ingrese un número de gran magnitud, un desbordamiento de pila o stack overflow pueda tener lugar, ya que el programa consumirá todo el espacio de memoria asignado. Esto se debe a que para cada n recibido como parámetro, se ejecuta la instrucción `addi $sp, $sp, -8`, creando un nuevo marco de pila de 8 bytes para contener su estado.

En cambio, la implementación iterativa tiene un consumo de memoria constante y puede expresarse como $O(1)$. El puntero de pila (`$sp`) no se manipula, ya que todas las operaciones aritméticas se realizan con el valor en el registro `$a0` y registros temporales (`$t0`), que se utilizan para

variar entre 0 y 1 (par, impar) y efectuar la operación de resta. Debido a esto, la versión iterativa no tiene riesgo de desbordamiento de memoria, siendo una implementación más segura.

En lo que corresponde al tiempo de ejecución, una vez más el enfoque iterativo destaca sobre el recursivo en cuanto a ciclos de reloj. Dado que en un procesador MIPS los registros internos de la CPU son más accesibles en términos de tiempo que la RAM, el algoritmo iterativo es más veloz en la ejecución a causa de que las operaciones que se realizan dentro del ciclo principal se llevan a cabo dentro de los registros lógicos y aritméticos. En la versión recursiva se escribe constantemente en la memoria RAM antes cada salto e incluso se lee cuando se deben recuperar los valores de la memoria. Esto, junto con el uso de jal y jr incrementa el tiempo de ejecución.

Para finalizar, con los factores previamente analizados se puede justificar la elección del enfoque iterativo para este problema en específico. La recursividad es recomendable para otro tipo de problemas donde se deben realizar búsquedas con backtracking o recorres estructuras de datos complejas como árboles, y no tanto en operaciones secuenciales como alternar entre par e impar que puede solucionarse con una función iterativa que se ejecuta en menor tiempo y no conlleva un consumo excesivo de memoria.

7. Análisis y Discusión de los Resultados.

Con la finalidad de examinar más a fondo y objetivamente el desempeño de los dos algoritmos de paridad tanto en la veracidad de las respuestas, tiempo de ejecución y consumo de memoria, además de realizar comparaciones con los datos obtenidos tras su simulación en el entorno MARS, se llevaron a cabo distintos casos de prueba, observando el comportamiento del procesador ante una implementación iterativa y recursiva.

Entre estos casos, se verificó el caso base de manera directa con un valor de $n = 0$, que es el menor valor posible permitido para determinar la paridad, con el propósito de comprobar que el algoritmo devuelva de manera exitosa el resultado y que haya lugar para ciclos infinitos con este caso borde o frontera. Para evaluar un número impar bajo y observar una ejecución corta paso a paso se eligió $n = 7$, un valor para el que el algoritmo no debería presentar problemas.

De igual manera, se realizaron pruebas con valores más altos para corroborar que los algoritmos permanezcan estables incluso con números que requieren una cantidad de llamadas e iteraciones mucho mayores a medida que se crea una mayor exigencia en los recursos del procesador, escogiendo $n = 752$ y $n = 12309$.

Tras varios intentos con números cada vez más altos ingresados con ayuda del simulador para crear intencionalmente un caso de falla, se pudo llegar a un valor máximo permitido para el algoritmo recursivo, sirviendo como otro caso borde. Este corresponde a $n = 523.774$, ya que se pudo observar en la consola que a partir de $n = 523775$ indica error. Dicha situación no ocurre para el algoritmo iterativo, para el cual no se halló un valor máximo.

Pasando a otras métricas, se determinó conveniente medir el tiempo de ejecución en cada prueba realizada. Por el motivo de que el simulador MARS trabaja con la ayuda del hardware del equipo en el que se esté desempeñando, no ofrece directamente la velocidad de un procesador MIPS32. Sin embargo, cuenta con una herramienta llamada Instruction Statistics que calcula el total de instrucciones que se realizaron en el programa con el valor dado.

De la misma manera, el simulador no tiene un oscilador de hardware independiente, por lo que para realizar el cálculo del tiempo de ejecución se consideró una frecuencia de 100 MHz (100×10^6 Hz), que es común a procesadores MIPS como el MIPS R2000. Asumiendo un CPI = 1 (un ciclo por instrucción), el tiempo de ciclo obtenido es

$$T = \frac{1}{f} = \frac{1}{100 \times 10^6 \text{ Hz}} = 1 \times 10^{-8} \text{ s} = 10 \text{ ns}$$

Con estos datos, fue posible utilizar la fórmula del tiempo de ejecución para cada uno de los

casos de prueba:

$$Tiempo = Instrucciones \times CPI \times Tiempo\ de\ Ciclo$$

Por otra parte, para evaluar el consumo de memoria también se efectuaron cálculos con la fórmula

$$Memoria(n) = 8 \times n\ (bytes)$$

, que permite obtener el tamaño en bytes de un marco de pila o stack frame, que el procesador debe reservar cada vez que se hace una llamada recursiva, en este caso, son 8 bytes ya que se debe guardar el valor de `n` (\$a0 (4 bytes) y la dirección de retorno \$ra (4 bytes).

Los resultados de cada caso de prueba se detallan en la siguiente tabla para comparar los dos enfoques:

Valor de entrada n	Resultado obtenido		Consumo de memoria		Tiempo de ejecución		N° de instrucciones	
	Iterativo	Recursivo	Iterativo	Recursivo	Iterativo	Recursivo	Iterativo	Recursivo
0	Par	Par	0 bytes	8 * 0 = 0 bytes	270 ns	320 ns	27	32
7	Impar	Impar	0 bytes	8 * 7 = 56 bytes	620 ns	1230 ns	62	123
752	Par	Par	0 bytes	8 * 752 = 6.016 bytes	37.870 ns	98.080 ns	3.787	9.808
12.309	Impar	Impar	0 bytes	8 * 12.309 = 98.472 bytes	615.720 ns	1.600.490 ns	61.572	160.049
523.774	Par	Par	0 bytes	8 * 523.774 = 4.190.192 bytes	26.188.970 ns	68.090.940 ns	2.618.897	6.809.094
523.775	Impar	Error	0 bytes	8 * 523.775 = 4.190.200 bytes	26.189.020 ns	36.664.350 ns	2.618.902	3.666.435

Con los datos brindados por cada caso de prueba, se puede afirmar que en todos los ensayos hasta el valor máximo admitido por la versión recursiva (523.774), ambos algoritmos ofrecieron el resultado adecuado de la paridad de cada valor de `n` ingresado. Por esta parte, ambas alternativas pueden resolver el problema planteado, pero con desigualdades que no pueden ser ignoradas en cuanto a eficiencia.

En el consumo de recursos, la variante iterativa es sin lugar a dudas la más rápida y eficiente en cada uno de los casos, ya que al no utilizar la pila, no reserva espacio en memoria para almacenar el valor de `n` y \$ra. Por el contrario, el algoritmo recursivo crece linealmente en lo que se refiere al consumo de recursos, creando tantos marcos de pila como el valor de `n`, cada uno reservando 8 bytes.

Es un gasto acumulativo de memoria que permite conocer el motivo de por qué la implementación recursiva tiene un límite mientras que la iterativa no, ya que mientras esta última no impacta la memoria, la recursiva puede llegar a sobrepasar sus límites y colapsar dependiendo de la magnitud de `n`, por lo que no solo es un problema de lento desempeño, sino que es una contingencia a nivel estructural. En este aspecto se concluye que siempre que el sistema disponga del tiempo de CPU suficiente, se podrá dar solución a valores mucho mayores con la iteratividad.

En otro orden de ideas, al estudiar los tiempos de ejecución calculados, se puede advertir que para números pequeños como `n = 7`, la diferencia de tiempo es de 610 ns (620 ns en el algoritmo

iterativo, 1230 ns en el recursivo). Es una disparidad que aunque no se note tan fácilmente por el usuario, es una enorme disminución en tiempo de procesamiento.

Llegando a situaciones extremas, con $n = 523.775$, se evidencia la gran pérdida de velocidad de respuesta que ocasiona la gestión de la pila, con un tiempo de ejecución de 26.188.970 ns del enfoque iterativo en comparación con los 68.090.940 ns del recursivo, siendo estos mayores al doble de nanosegundos del tiempo iterativo. Con la lectura y escritura en la RAM, que bien son necesarias, tienen peso en el rendimiento del programa.

Otro punto a destacar es que en el enfoque iterativo solo se hacen saltos condicionales con beq y saltos incondicionales con la instrucción j, mientras que en el algoritmo recursivo se deben ejecutar instrucciones como jal y jr para cada valor de n , que deben actualizar una y otra vez el valor del registro \$ra y escribir en memoria para preservar la dirección anterior, aumentando el número de instrucciones que pueden verse en el conteo total, que resultan en un mayor tiempo de ejecución y consumo de memoria.

Por otro lado, en el caso de prueba con $n = 523.775$ la recursividad produce un desbordamiento de pila o stack overflow. Cuando esto ocurre, MARS detiene la ejecución de manera repentina ante este imprevisto, por lo que el tiempo de ejecución que se pudo calcular no equivale al tiempo total requerido para dar solución al problema con este valor, tomando en consideración que en la prueba con $n = 523.774$ el tiempo es de 68.090.940 ns y el que pudo calcularse hasta que se detuvo la ejecución fue de 36.664.350 ns. Una conclusión derivada de esto es que el valor ingresado o n debe validarse antes de empezar a realizar las operaciones para advertir al usuario del límite de la versión recursiva además de evitar el consumo de memoria y trabajo en los recursos para casos donde no es posible llegar a una solución por impedimentos de hardware.

A modo de cierre, podemos decir que los resultados permiten constatar que para este ejercicio en lenguaje ensamblador MIPS32 y su respectivo procesador, la versión iterativa es la más indicada, siendo conveniente al proporcionar un rendimiento superior al del enfoque recursivo y garantizar la correcta solución para la paridad de cualquier número entero positivo sin dar lugar a errores como el desbordamiento de pila. A pesar de la similitud en cuanto a la lógica matemática del algoritmo recursivo, esta implementación exige abundante almacenamiento en la memoria para cada marco de pila y el tiempo de ejecución ofrecido no es menor que el de la alternativa con iteratividad incluso para el valor de n más pequeño que pueda ingresarse.